

**Keresési feladat: feladatrepresentáció, vak keresés, informált keresés, heurisztikák.**  
**Kétszemélyes zero összegű játékok: minimax, alfa-béta eljárás. Korlátozás kielégítési feladat**

## KERESÉSI FELADATOK

A feladatok célja, hogy meghatározzanak egy optimális cselekvéssorozatot egy adott probléma megoldásához. Egy keresési feladat során egy ágensnek kell a lehetőségei közül választania, hogy egy kezdeti állapotból egy kívánt célállapotba jusson

### Feladatrepresentáció

- **Ágens:** valami ami cselekszik
  - **Racionális ágens:** egy olyan ágens, amely a tudásához viszonyítva helyesen cselekszik
  - **Problémamegoldó ágens:** olyan ágens, mely úgy határozza meg, hogy mit is kell tennie, hogy olyan cselekvéssorozatokat keres, mely a kívánt célállapotokba vezetnek (célorientált ágens)
- vizsgált problémák esetén feltesszük, hogy a **feladatkörnyezet:**
  - **diszkrét:** az állapotok és az időkezelés nem folytonosak, hanem jól elkülöníthető egységekből állanak
  - **statikus:** a környezet változatlan amíg az ágens gondolkodik
  - **teljesen megfigyelhető:** az ágens a szenzorjai segítségével a környezet teljes állapotát ismeri
  - **determinisztikus:** a környezet következő állapotát a jelenlegi állapota és az ágens által végrehajtott cselekvés teljesen meghatározza (egy cselekvés kimenete egyértelmű)
- modell
  - ha a fenti feltételek adottak akkor a problémák ezekkel modellezhetőek:
    - **lehetséges állapotok halmaza:** a világ minden olyan konfigurációja, amely előfordulhat
    - **kezdőállapot:** az ágens innen kezdi a cselekvést
    - **lehetséges cselekvések halmaza:** legáltalánosabb leírása az állapotátmenet függvény. Egy  $x$  állapothoz hozzárendel egy  $\langle \text{cselekvés}, \text{utódállapot} \rangle$  párok halmazát, ez megadja, hogy az adott állapotban milyen cselekvést végezhetünk és az milyen utódállapotba vezet
    - **állapotátmenet költségfüggvénye:**  $\langle \text{állapot}, \text{cselekvés}, \text{utódállapot} \rangle$  háromashoz hozzárendel egy valós számot (költség)
    - **célállapotok halmaza:** a lehetséges állapotok részhalmaza
  - a modell egy **irányított, súlyozott gráfot** definiál, másnéven **állapottér**
    - a gráf csúcsai az állapotok
    - élek a cselekvések
    - élek súlyai a cselekvések költségei
    - az út az állapotok és cselekvések olyan sorozata, mely a kezdőállapotból egy célállapotba vezet

## Algoritmusok hatékonyságának mérőszámai

- 4 szempont van: teljesség, optimális, idő- és tárigény
- **teljesség:** garantálja-e az algoritmus a megoldást ha az létezik?
  - algoritmus teljes, ha minden esetben létezik véges számú állapot érintésével elérhető célállapot és az algoritmus legalább egy ilyen meg is talál
- **optimális:** mindig a legkisebb költségű utat találja meg?
  - algoritmus optimális, ha teljes és minden talált célállapot optimális költségű
- **idő- és tárigény**

## Vak keresés (informálatlan)

- a problémák modellje egy irányított súlyozott gráfot definiál (állapottér), ahol a csúcsok az állapotok, az élek cselekvések, a súlyok pedig a költségek
- semmilyen információ nincs az állapotokról, csak annyi amennyi a probléma definíciójában szerepel, csak a generált állapotokat ismeri
- az ágensek feladata az, hogy adott kezdőállapotból találjanak egy minimális költségű utat egy célállapotba
  - ez az állapottérben végrehajtott kereséssel történik, de nem egészen a klasszikus legrövidebb út keresési probléma
    - az állapottér nem mindig adott explicit módon, és végtelen is lehet
- pl. Térkép: állapotok=városok; cselekvés=úttal összekötött két város közti áthaladás; költség=távolság; a kezdő és célállapot feladatfüggő. A probléma itt útvonaltervezés, a térkép pedig a világ modellje
- **Keresőfa**
  - keresőfa növesztése a kezdő állapotból
    - keresőfa nem feltétlen azonos a feladat állapottérével, mivel az nem biztos, hogy fa, lehet akár gráf is, illetve végtelenre is nőhet akkor is ha az állapottér véges
      - végtelenre nő, az azt jelenti, hogy végtelen ciklusba kerülhet, például ha körök vannak benne (gráf)
  - csomópontok reprezentációja
    - **Állapot:** az állapottérnek a csomópontához tartozó állapota
    - **Szülő-csomópont:** a keresési fa azon csomópontja, amely a kérdéses csomópontot generálta
    - **Cselekvés:** a csomópont szülő-csomópontjára alkalmazott cselekvés, tehát az az operátor amellyel a szülőből az adott állapotba jutottunk
    - **Út-költség:** a kezdeti állapotból az adott csomópontig vezető út általában  $g(n)$ -nel jelölt költsége
    - **Mélység:** a kezdeti állapotból vezető út lépéseinek a száma

- **Fakeresés**

```
fakeresés
1 perem = { újcsúcs(kezdőállapot) }
2 while perem.nemüres()
3     csúcs = perem.elsőkivesz()
4     if csúcs.célállapot() return csúcs
5     perem.beszúr(csúcs.kiterjeszt())
6 return failure
```

- perem (nyílt halmaz), egy prioritási sor, ami a már legenerált de még kifejtésre váró csomópontokat tárolja
  - keresési stratégiáját az *elsőkivesz()* függvény valósítja meg
    - hatékonyságát befolyásolja, illetve más más megvalósításokkor az eredmény is más lehet (pl: a sorrend, legutolsó kivevése stb)
- *csúcs.kiterjeszt()* generálja az aktuális állapotból elérhető állapotok halmazát

- **Gráfkeresés**

```
gráfkeresés
1 perem = { újcsúcs(kezdőállapot) }
2 zárt = {}
3 while perem.nemüres()
4     csúcs = perem.elsőkivesz()
5     if csúcs.célállapot() return csúcs
6     zárt.hozzáad(csúcs)
7     perem.beszúr(csúcs.kiterjeszt() - zárt)
8 return failure
```

- végtelen ciklusok elkerülése miatt a fakereséstől eltérően itt perem (nyílt halmaz) mellett, **zárt halmazt** is használ a végtelen ciklus elkerüléséért
  - ha nem fa az állapottér hanem gráf akkor lehet benne kör ami miatt végtelen ciklus, ezt zárja a zárthalmaz ki
  - a **zárt halmaz a már kiterjesztett csúcsokat tárolja**, és a perembe csak olyanokat veszünk be ami még nem tagja a zárt halmaznak
- **Szélességi keresés**
  - szintről szintre halad
  - FIFO (First In First Out) peremmel (verem) valósítja meg a gráfkeresést
    - pl gyökér elem 2 gyereket berakjuk jobbról balra, vagy abc sorrendbe akkor azt terjesztjük ki előbb ami elsőnek került be
  - idő és tárigény:  $O(b^{d+1})$ 
    - $b$  (elágazási tényező),  $d$  (legsekélyebb cél mélysége)
  - teljes és általában nem optimális
- **Mélységi keresés**
  - lemegy a fába amilyen mélyre csak lehet, majd ha bejárta visszalép (backtracking)
  - LIFO (Last In First Out) peremmel (sor) valósítja meg a gráfkeresést
    - pl azt terjesztjük ki előbb ami később került be

- egyszerre 1 gyerek kerül be kiterjesztéskor a perembe
- időigény:  $O(b^m)$  - rosszabb mint a szélességi
- tárigény:  $O(b * m)$  - jobb mint a szélességi
  - $b$  (elágazási tényező),  $m$  (fa maximális mélysége)
- teljes és nem optimális
- **Iteratíván mélyülő keresés (legjobb)**
  - mélységi keresések sorozata 1, 2, 3, ..., n mélységre
    - korlátozott mélységig
  - bekerül mindig egyszerre az összes gyerek a kiterjesztéskor
  - teljes és optimális
  - legjobb vak kereső
    - egyesíti a mélységi keresés alacsony tárigényét a szélességi keresés teljességével és optimalitásával
- **Egyenletes költségű keresés**
  - először a peremből a legkisebb útköltségűt terjesztjük ki
    - NEM él hanem út költség, tehát mindig eltároljuk az adott csúcsig megtett utat
      - ezért figyelni kell, hogy ismétlődés esetén előbb utóbb egy másik élre esik a választás
  - az idő- és tárigénye nagyban függ a költségfüggvénytől
  - teljes és optimális, ha pozitívak az élek ( $él > 0$ )
  - tulajdonképp a Dijkstra algoritmus alkalmazása az állapottéren

## Informált (heurisztikus) keresés

- van egy célunk, azaz tudjuk hová megyünk
  - háttértudás, heurisztika (pl. légvonalbeli távolság) is kell, ez egy heurisztikus függvénnyel valósítjuk meg, amely a legolcsóbb utat becsli meg
- **Legjobb először**
  - fa vagy gráfkeresés algoritmusok speciális esete, ahol egy csomópont kifejtésére való kiválasztása az  **$f(n)$  kiértékelő függvény**től függ
    - általában az a csomópont kerül kiértékelésre egy lépésben mely a legkisebb értéket adja vissza (tehát az adott pillanatba a legjobbnak tűnőt)
  - ez egy általános megközelítés, több változata létezik amik a kiértékelő  $f(n)$  függvény megvalósításában különböznek: mohó legjobb először és az  $A^*$
  - ebbe a családba tartozó algoritmusok egy  **$h(n)$  heurisztika függvény**t használnak, amely megbecsli az adott csomópontból a célig vezető legolcsóbb út költségét
    - nem pontos érték csupán becslés
    - minden problémánál más heurisztika a legjobb pl: légvonalbeli távolság (útvonal tervezési probléma esetén)

- **Mohó legjobb először**

- perembe  $f(n) = h(n)$  alapján vannak a csomópontok rendezve és a legkisebb értékű csúcsot vesszük ki a következő lépésbe (tehát ami legközelebb van a célhoz)
- teljes, de csak akkor ha keresési fa véges mélységű
- nem optimális
- időigény = tárigény =  $O(b^m)$ 
  - legrosszabb eset nagyon rossz, de jó  $h(n)$  heurisztika választással javítható
  - $m$  a keresési fa mélysége (maximális lépés szám, amelyet az algoritmus előre néz)
  - $b$  az elágazási faktor (egyes csúcsokból mennyi lehetséges lépés van)

- **A\* algoritmus**

- a  $h(n)$  heurisztika függvényen kívül használt egy  $g(n)$  függvényt is, mely az aktuális csúcsig  $(n)$  megtett út költségeit adja vissza (tényleges megtett út)
- a perembe a csomópontok  $f(n) = g(n) + h(n)$  függvény szerint vannak rendezve és a következő lépésben azt a csúcsot veszi ki amelyiken keresztül vezető legolcsóbb megoldás becsült költsége a legkisebb
- fa-keresés esetén optimális és teljes, ha a keresési fa végés és  $h(n)$  heurisztika elfogadható
  - heurisztika akkor elfogadható ha soha nem becsüli felül a cél eléréséhez szükséges költséget
- gráf-keresés esetén optimális és teljes, ha az állapottér véges és a  $h(n)$  heurisztika konzisztens
  - heurisztika akkor konzisztens ha  $h(n) \leq c(n; a; n') + h(n')$  tetszőleges  $n$ -re és annak tetszőleges  $n'$  szomszédjára
    - ahol  $c(n, a, n')$  az  $n$  állapotból az  $n'$  állapotba való átlépés tényleges költsége
  - bár létezik olyan változata is az algoritmusnak mikor a konzisztencia nem lényeg (itt a már bejárt (zárt halmazba lévő) csúcsokat át lehet linkelni)
- tárigény exponenciális, de függ a  $h(n)$  minőségétől
  - pl. ha  $h(n) = h^*(n)$ , ahol  $h^*(n)$  a tényleges hátralevő költség, akkor konstans
- az A\* optimálisan hatékony, tehát egyetlen más optimális algoritmus sem fejt ki garantáltan kevesebb csomópontot, mint az A\*
- módosított változata az **egyszerűsített memóriakorlátozott A\***
  - ameddig van memória fut az A\*, ha elfogyott töröljük a legrosszabb levelet (több ilyen van akkor a legrégebbet)
    - hibája, hogy hasonló költségű utak esetén ugrálhat ezek között (eldobja, majd újraépíti őket)
  - törlés, eldobás után a törölt csúcs szülőjében mentésre kerül az innen elérhető legjobb ismert költség

- nem optimális, nem találja meg az optimális megoldást, ha nincs elég memória

## Heurisztikák

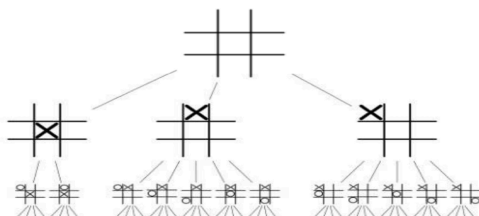
- kombinatorikus optimalizálási problémák esetében gyakran szükséges, hogy algoritmusunk egy része, ahol a döntési részek vannak, végignézi a lehetséges döntési kimeneteket
  - ez általában exponenciális, az input mérettől függ, de kis inputok esetén is már hasznavehetetlen
- a heurisztikák olyan probléma / terület specifikus tudást adnak, amelyek segítenek a keresési tér gyorsabb és hatékonyabb bejárásában, mint az informálatlan keresési algoritmusok
  - a heurisztikus megközelítések célja, hogy minimalizálják a keresési tér méretét azáltal, hogy csak a legígéretesebb útvonalakat vizsgálják meg (elkerülik a szükségtelen csúcsok kifejtését)
- a heurisztikus kiértékelésre teljesülnie kell, hogy:
  - a végállapotokra az eredeti értéket adja
  - gyorsan kiszámolható
  - nem végállapotokra a nyerési esélyt jól tükröző érték
    - finomabb felbontás mint a minimax értékek (ami gyakran 1 vagy -1)
- heurisztikus függvény előállítása
  - **relaxált probléma optimális megoldása**
    - relaxálás = feltételek elhagyása, azaz könnyebbé tesszük a problémát
    - a relaxálás az állapottérről szól
      - új probléma definiálása, mely az eredetihez új átmeneteket és / vagy célállapotot ad hozzá
    - relaxált probléma optimális költsége  $\leq$  mint az eredeti probléma optimális költsége
      - elfogadható a heurisztika
    - példa: a tologatós 8-kirakó, adjunk ehhez pár heurisztikák
      - alap probléma feltétele: csak szomszédos üres helyre tudunk tolni négyzeteket
      - $h_1()$ : első feltétel elhagyása (most már bármilyen üres mezőre tudunk tenni)
      - $h_2()$ : második feltétel elhagyása (most már a nem üres mezőkre is lehet tolni)
        - ilyenkor elég kiszámolni a rossz helyen lévő négyzetek Manhattan-távolságát (a szükséges függőleges és vízszintes lépések száma) a megfelelő helyüktől és a kapott értékeket összeadni (így az ábrára nézve  $h_2() = 18$ )
      - $h_{opt}() = 26$ , az ábrára nézve
      - első relaxációnk tetszőleges kirakósra mindig kisebb értéket fog generálni, mint a második (nyilván a szomszédos mezőre rakás jobban megnehezíti ezt a feladatot, mint a négyzet üressége), ezért azt mondjuk, hogy  $h_2$  dominálja  $h_1$ -et
      - mivel az eredeti probléma optimális költségétől a relaxált probléma optimális költsége  $\leq$  ezért elfogadható a heurisztika

|   |   |   |
|---|---|---|
| 7 | 2 | 4 |
| 5 |   | 6 |
| 8 | 3 | 1 |

- Keresés levágása
  - heurisztikus kiértékelést így tudjuk kihasználni:
    - **Fix mélység**
      - az aktuális játékalrásból pl.  $X$  mélységig építjük fel a keresőfát alfa-bétával, ami során az  $X$  mélységű állapotokat értékeljük ki heurisztikusan. Az implementáció egyszerű, kb ugyanaz csak a mélységet is követni kell, és végállapotok mellett az  $X$  mélységre is tesztelünk a rekurzió előtt
    - **Iteratívan mélyülő keresés**
      - alfa-béta iteráltan növekvő mélységig. Ez analóg a korábbi iteratívan mélyülő kereséssel (az alfa-béta pedig a mélységi kereséssel). Ennek a legnagyobb előnye, hogy rugalmas: már nagyon gyorsan van tippünk következő lépésre, és ha van idő, akkor egyre javul, tehát valós időben használható

## KÉTSZEMÉLYES ZÉRÓ ÖSSZEGŰ JÁTÉKOK

- fő érdeklődési területe az MI-nek (sakk motiválta, go játék még megoldatlan)
- ilyen problémák modellje
  - **lehetséges állapotok halmaz**a: legális játékalrások
  - **kezdőállapot**
  - **állapotátmenet függvény**: minden állapothoz hozzárendel egy  $\langle cselekvés, állapot \rangle$  típusú rendezett párokból álló halmazt (operátorok, azaz lehetséges lépések melyek új állapotba visznek)
  - **végállapotok** (célállapotok) **halmaz**a: lehetséges állapotok részhalmaz
  - **hasznosságfüggvény** (jutalomfüggvény): minden lehetséges végállapothoz hasznosságot rendel (pl győzelem = 1, vereség = -1)
    - egyik játékos maximalizálni, másik minimalizálni akarja
- az ilyen problémák modelje egy irányított gráfot definiál (játékgráf)
  - általában nem fa
- az nem számít milyen módon jutunk végállapotba
- a következőkbe a vizsgált játékokról feltesszük, hogy:
  - **kétszemélyesek**: 2 ágens van
  - **lépésváltásosak**: a 2 ágens felváltva lép (operátort alkalmaznak)
  - **determinisztikusak**: a lépés nem a véletlentől függ, hanem attól, hogy a játékos milyen lépést választ
  - **zéró összegűek**: a MAX játékos pontosan annyit nyer amennyit a MIN veszít
    - a hasznosságfüggvényt az egyik maximalizálni (MAX játékos), a másik meg minimalizálni akarja (MIN játékos), konvenció szerint MAX kezd
    - MIN hasznossága a MAX hasznosságának mínusz egyszerese



## Minimax

- **tökéletes racionalitás hipotézis** fenn kell álljon
  - a játékosok ismerik a játékgráfot teljesen, nem hibáznak és komplex számításokat is tudnak végezni
- rekurzív mélységi keresés a játékfaban, meghatározza az optimális lépést, feltételezve, hogy az ellenfél is optimálisan játszik
  - feltétel, hogy mind két játékosra teljesül a tökéletes racionalitás hipotézise

$$\text{minimax}(n) = \begin{cases} \text{hasznosság}(n) & \text{ha végállapot} \\ \max_{a \in n \text{ szomszédai}} \text{minimax}(a) & \text{ha MAX jön} \\ \min_{a \in n \text{ szomszédai}} \text{minimax}(a) & \text{ha MIN jön} \end{cases}$$

Az algoritmus:

|                                                                                                                                                             |                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>maxÉrték(n) 1 if végállapot(n)   return hasznosság(n) 2 max &lt;- -végtelen 3 for a in n szomszédai 4   max = max(max, minÉrték(a)) 5 return max</pre> | <pre>minÉrték(n) 1 if végállapot(n)   return hasznosság(n) 2 min &lt;- +végtelen 3 for a in n szomszédai 4   min = min(min, maxÉrték(a)) 5 return min</pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|

- minimax értékek
  - a teljes játékfa ki kell számítani a minimax értékeket
    - tehát a levelekig mindenképp le kell menni mivel onnan indul visszafelé és meghatározza az értéküket a csomópontoknak
  - a gyökérben lévő minimax érték a legjobb elérhető érték egy másik TÖKÉLETES játékos ellen
  - optimális megoldás csak a kezdőállapotból indulva érhető el
- menete (általában a MAX kezd, ez a konvenció)
  - a játékos kiszámítja a minimax értékek az egész játékfa
  - minden lépésnél a számára legjobb minimax értékű szomszédot lépi (ez a stratégia)
- optimális, de időigénye exponenciális ( $O(b^m)$ ), így teljes játékfanál nem futtatható végig, a  $b$  (elágazási tényező), az  $m$  (fa maximális mélysége)
- ha a játékgráfba, kör van akkor nem terminál, de gyakorlatban nem probléma, mert csak fix mélységig futtatjuk, illetve a játékszabályok is gyakran kizárják a körök lehetőségét a játékgráfba
- **Alfa-béta eljárás (vágás)**
  - *főkérdése*: mivel például sakk esetén 10154 csúcs van a játékfaban amiből 1040 különböző, erre hatékonyan, hogy érdemes minimaxot számolni?
    - a játékban a vizsgálandó állapotok száma exponenciális a lépések számában
  - az alfa-béta eljárás lényege, hogy megfelezi a kitevőt, mivel nem kell bejárni a teljes fát
    - bejárás módjától, azaz attól, hogy hogyan járjuk be a gyerek csúcsokat függ mennyi részét kell bejárni az egész játékfanak



- a keresési fa bizonyos ágait levágjuk (nem vizsgáljuk meg), ha bebizonyosodik, hogy azok nem befolyásolják a végső döntést
- ötlet: ha tudjuk, hogy például MAX már ismer legalább egy olyan stratégiát, amellyel ki tud kényszeríteni például legalább 10 értékű hasznosságot egy adott állapotból indulva, akkor az adott állapot alatt nem kell vizsgálni olyan állapotokat, amelyekben MIN ki tud kényszeríteni  $\leq 10$  hasznosságot, hiszen tudjuk, hogy MAX sosem fogja ide engedni a játékot

|                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> maxÉrték(n, alfa, béta) 1 if végállapot(n) return   hasznosság(n) 2 max &lt;- -végtelen 3 for a in n szomszédai 4   max = max(max, minÉrték(a,     alfa, béta)) 5   if max&gt;=beta return max 6   alfa = max(max, alfa) 7 return max </pre> | <pre> minÉrték(n, alfa, béta) 1 if végállapot(n) return   hasznosság(n) 2 min &lt;- +végtelen 3 for a in n szomszédai 4   min = min(min, maxÉrték(a,     alfa, beta)) 5   if alfa&gt;=min return min 6   beta = min(min, beta) 7 return min </pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- működése

#### ■ 2 új paraméter

- **alfa**: biztos, hogy MAX játékos ki tudja kényszeríteni, hogy a hasznosság legalább alfa legyen, valamely olyan állapotból indulva, ami  $n$  állapotból a gyökér csúcs felé vezető úton van. Azaz ez **egy minimum érték amit a MAX már biztosított magának**

- kezdéskori érték -végtelen

- **béta**: biztos, hogy MIN játékos ki tudja kényszeríteni, hogy a hasznosság legfeljebb béta legyen, valamely olyan állapotból indulva, ami  $n$  állapotból a gyökér csúcs felé vezető úton van. Azaz ez **egy maximum érték amit a MIN már biztosított magának**

- kezdéskori érték +végtelen

- ha a MAX kezd, akkor a *maxÉrték(kezdő\_csúcs, -végtelen, +végtelen)* kerül meghívásra először, ha a min akkor a *minÉrték*

- ezután a fa rekurzív vizsgálata következik és eközben az alfa és béta értékek folyamatosan módosulnak

- a csúcsok kezdetben öröklik a szülő csúcs alfa és béta értékeit
- 2 féle csúcs van, van MIN és MAX, fa forma esetén mélységenként változik, ha a MAX kezd akkor az első sor MAX, majd következő MIN és így felváltva

- **alfa frissítés**

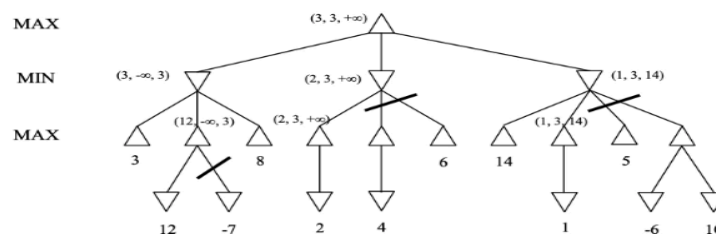
- adott csúcs gyerekeinek vizsgálata során ha MAX csomópontba a gyerek minimax érték nagyobb, mint az aktuális alfa akkor, frissítjük a gyerek minimax értékére

- **béta frissítés**

- az adott csúcs gyerekeinek vizsgálata során ha MIN csomópontba a gyerek minimax értéke kisebb mint az aktuális béta akkor, frissítjük a gyerek minimax értékére

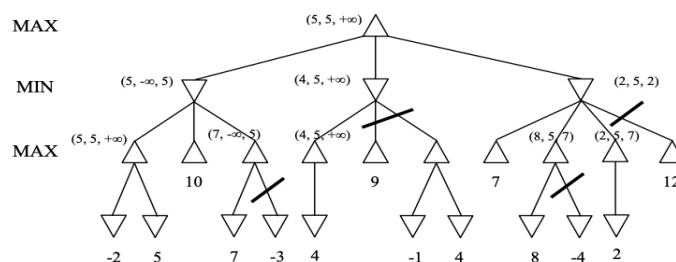
- mikor már nincs értelme tovább vizsgálni, mert nem állhat elő egy állás akkor az adott csúcs utódai levágásra kerülnek
  - gyökér csúcs alatt közvetlen nincs vágás
  - **alfa vágás**
    - ha MIN játékos esetén az aktuális minimax érték kisebb vagy egyenlő mint az aktuális alfa, akkor az azt jelenti, hogy ez az állás nem állhat elő
  - **béta vágás**
    - ha MAX játékos esetén az aktuális minimax érték nagyobb vagy egyenlő mint az aktuális béta akkor tudjuk már, hogy ez az állás nem állhat elő
- tetszőleges állaptoból indítva nem áll végig fenn a minimax stratégia a játék végéig (csak gyökér csúcsból indulva áll fenn végig a minimax stratégia)

#### 1. feladat



Az ábrán a levél csomópontok a végállapotok, az alattuk szereplő értékek a hozzájuk tartozó hasznosságértékek (amik így egyben minimax értékek is). Az algoritmus a játékgáf csomópontjait (a rekurzív függvényhívások miatt) a mélységi keresésnek megfelelő sorrendben járja be, a csomópontokhoz tartozó minimax, alfa és béta értékeket közben folyamatosan frissítve. A nem levél csomópontok mellett szereplő értékek az algoritmus teljes futása után a csomópontokhoz rendelt (minimax, alfa, béta) hármasok.

#### 2. feladat



## KORLÁTOZÁS KIELÉGÍTÉSI FELADAT

- cél olyan teljes hozzárendelést találni, ahol minden változóhoz tartozik érték és az összes feltétel teljesül (konzisztens)
  - tehát cél olyan állapotokat találni, ami megfelel bizonyos feltételeknek
    - pl: gráf színezési probléma esetén, hogy a gráf csúcsait különböző színekkel kell színezni úgy, hogy két szomszédos csúcs ne legyen azonos színű
  - **konzisztens (megengedő) hozzárendelés:** a hozzárendelés nem sért meg egyetlen korlátozást sem
  - **teljes hozzárendelés:** minden változóhoz van érték
  - út lényegtelen a megoldásig
- algoritmusok két fő típusa
  - **inkrementális keresés:** lépésről lépésre csökkentjük az ismeretlen értékek számát
  - **optimalizáló keresés:** megsértett korlátozások számát minimalizálja
- ilyen KKF feladatok az állapottérrel adott keresési és optimalizálási problémák jellemzőit ötvözi, ahol az állapotok és célállapotok speciális alakúak
  - változók halmaza:  $(X_1, \dots, X_n)$
  - lehetséges állapotok halmaza (domain-ek)
    - $D = D_1 \times \dots \times D_n$ , ahol  $D_i$  az  $i$ -edik változó lehetséges értékei, azaz a feladat állapotai az  $n$  darab változó lehetséges értékkombinációi
  - célállapotok
    - a megengedett állapotok, amelyek definíciója a következő: adottak  $C_1, \dots, C_m$  korlátozások,  $C_i \subseteq D$ . A megengedett vagy konzisztens állapotok halmaza a  $C_1 \cap \dots \cap C_m$  olyan állapotokat tartalmaz, amelyek minden korlátozást kielégítenek
- példák
  - gráf színezés
    - lehetséges állapot  $(1, 1, 1), (1, 2, 1) \dots (2, 2, 2)$
    - viszont a végén két azonos szín nem lehet egymás mellett (cél állapotok, korlátozások):  $(1, 2, 1), (2, 1, 2)$
  - 8-királynő probléma

### Inkrementális kereső algoritmusok (backtracking)

- keresési + optimalizálási problémák jellemzőit ötvözik, de úgy álljunk hozzá mint a keresési problémákhoz
  - mélységi keresés egy változata
- minden inkrementációban próbáljuk csökkenteni az ismeretlenek számát, ezáltal közelítünk a megfelelő állapothoz

- állapotér:
  - minden változóhoz felvesszünk egy új ismeretlen értéket ("??")
    - kezdőállapot: (?, ..., ?)
  - az állapotátmenet függvény minden állapothoz azokat az állapotokat rendeli ahol eggyel kevesebb "?" van és amelyek még megengedettek
  - a költség minden állapotra konstans
- gráf / fa áll elő
  - körmentes és véges (annyi a mélysége ahány változó van)
- teljes és optimális (ha van út megtalálja, ha nincs akkor ezt kapjuk meg)
  - de nem skálázódik jól nagy problémákra
- bármilyen informálatlan algoritmust választhatunk a kereséshez
  - informált (heurisztikus) kereső algoritmus itt nem biztos, hogy talál utat

### Optimalizáló algoritmusok (lokális keresők)

- optimalizálással történik, itt a célfüggvény a megsértett korlátozások számát adja vissza és ezt szeretnénk minimalizálni (0-ra)
  - ha már van eredetileg célfüggvény akkor össze kell vele kombinálnunk
  - nem kell új állapotért létrehozni
- operátorokat sokféleképp lehet definiálni, például valamelyik változó értékenek megváltoztatásaként
  - ehhez használható, az összes korábban látott lokális kereső, genetikusan algoritmus stb
- nem mindig talál megoldást (nem optimális) és nem is teljes
  - de gyorsak és sikeresek, a megoldásig az út lényegtelen
- például hegymászó keresés
  - legegyszerűbb lokális keresés
  - egy ciklusba mindig felfelé lép az aktuális állapot szomszédai közül, lokális optimumba megragad
- ide tartozik még a branch and bound

#### Összegzés: Mikor melyiket?

- **Inkrementális + Konzisztencia (Backtracking + AC-3):** Akkor jó, ha a kényszerek szigorúak (*hard constraints*), kevés a megoldás, és biztosan meg akarjuk találni az egyiket, vagy be akarjuk látni, hogy nincs megoldás.
- **Lokális Keresés (Min-conflicts):** Óriási állapottér esetén, ahol a kényszerek lazábbak, vagy könnyű találni egy majdnem jó kiinduló állapotot.
- **Optimalizáló algoritmusok (Branch & Bound, Metaheurisztikák):** Ha a kényszerek teljesítése mellett egy számszerűsíthető minőséget (pl. költség, idő, távolság) is optimalizálni kell.

## Kényszergráf

- vizuálisan reprezentálja a Korlátozás kielégítési feladatokat egy gráf formájában
  - csúcsok a változók
  - élek a változók közötti bináris korlátozásokat (pl: színezésnél a szomszédokat) jelölik
- változók felett gráfot lehet definiálni
  - bármely kettőnél több változót érintő korlátozás kifejezhető változó párokon értelmezett korlátozásként, ha megengedjük a segédváltozók bevezetését
- minden feladathoz hozzáadható és a keresés komplexitásának a vizsgálatakor hasznos, mert a gráf tulajdonságainak függvényében adható meg a komplexitás

5.1. ábra - (a) Auszália államai és területei. A térkép kiszínezése tekinthető kényszerkielégítési problémának is. A cél az, hogy minden egyes részhez olyan színt találjunk, amely nem azonos egy szomszédos rész színével sem. (b) A térképszínezési probléma kényszergráfként reprezentálva.

